

Cell as Point: One-Stage Framework for Efficient Cell Tracking

Yaxuan Song^a, Jianan Fan^{a,*}, Heng Huang^b, Mei Chen^c, Weidong Cai^{a,*}

^a*School of Computer Science, The University of Sydney, Sydney, 2006, NSW, Australia*

^b*Department of Computer Science, University of Maryland College Park, College Park, 20742, Maryland, United States*

^c*Microsoft, Redmond, 98052, Washington, United States*

Abstract

Conventional multi-stage cell tracking approaches rely heavily on detection or segmentation in each frame as a prerequisite, requiring substantial resources for high-quality segmentation masks and increasing the overall prediction time. To address these limitations, we propose **CAP**, a novel end-to-end one-stage framework that reimagines cell tracking by treating **Cell as Point**. Unlike traditional methods, CAP eliminates the need for explicit detection or segmentation, instead jointly tracking cells for sequences in **one stage** by leveraging the inherent correlations among their trajectories. This simplification reduces both labeling requirements and pipeline complexity. However, directly processing the entire sequence in one stage poses challenges related to data imbalance in capturing cell division events and long sequence inference. To solve these challenges, CAP introduces two key innovations: (1) adaptive event-guided (AEG) sampling, which prioritizes cell division events to mitigate the occurrence imbalance of cell events, and (2) the rolling-as-window (RAW) inference strategy, which ensures continuous and stable tracking of newly emerging cells over extended sequences. By removing the dependency on segmentation-based preprocessing while addressing the challenges of imbalanced occurrence of cell events and long-sequence tracking, CAP demonstrates promising cell tracking performance and is **8 to 32 times more efficient** than existing methods. The code and model checkpoints are available at <https://github.com/YXSong000/CAP>.

*Corresponding authors.

Keywords: Cell tracking, Point tracking, RAFT, End-to-end one-stage framework

1. Introduction

Cells are the fundamental units of organs in the human body [1]. Cell events are diverse, continuously changing, and intricate, involving cell migration (translocation), mitosis (division), and apoptosis (death) [2]. Cell tracking is the task of keeping track of each cell’s location and reconstructing cell lineages (mother-daughter relations), which help develop preventive therapies [3] and drug discovery [4]. Traditional methods of manual tracking demand significant resources and expertise to achieve reliable results. These challenges highlight the necessity for developing efficient and automated cell tracking systems.

Several existing works and platforms [5, 6] have demonstrated effective performance in achieving automated cell tracking. The taxonomies of strategies typically involve segmentation followed by linking (SegLnk) [7], segmentation and linking (Seg-&Lnk) [8], or even detection prior to the segmentation and linking stages (DetSegLnk [9] or DetLnkSeg [2]) [10]. Also, frameworks [11, 12] often incorporate isolated data preprocessing stages to represent cell tracking data. These multi-stage cell tracking pipelines can limit the overall performance of the framework, constrained by the errors that occur at earlier stages and are accumulated and amplified at subsequent stages [13]. Additionally, a multi-stage framework involving both detection/segmentation and tracking not only requires the tracking ground truth but also demands additional labels, i.e., high-quality segmentation masks, to train the model. It is highly labor-intensive and time-consuming to produce the data annotation, rendering such methods impractical for clinical applications.

To address the limitations of existing multi-stage frameworks, we propose a novel **one-stage** cell tracking framework, **Cell as Point** (abbreviated as **CAP**), as demonstrated in Figure 1 (a), that treats each cell as a point to jointly track numerous cell points within a sequence. The one-stage CAP framework not only benefits from the efficient prediction procedure through a straightforward pipeline as shown in Figure 1 (b), but also from its low requirement for data annotation, solely relying on the track-

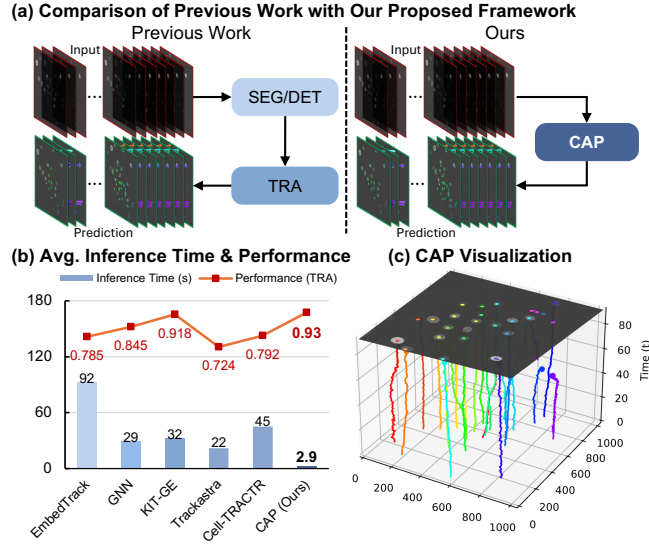


Figure 1: Our proposed **CAP** is an end-to-end trained framework leveraging the idea of *Cell as Point* to facilitate tracking cells efficiently. As (a) illustrates, instead of previous work requiring segmentation (SEG) or detection (DET) as a prerequisite for final tracking (TRA), CAP is able to track all cells within sequence frames in **one stage**. (b) shows that CAP reduces the inference time (2.9s) by approximately **8 to 32 times** compared to previous works while maintaining high tracking performance of 0.93. (c) demonstrates the cell tracking result predicted by CAP.

ing ground truth (coarse masks and cell acyclic graphs) to represent the position and status of each cell. Specifically, by proposing **cell point trajectory** and **visibility** to represent the cell tracking data within a one-stage framework comprehensively, CAP offers an elegant solution to alleviate the issues of the complicated data preprocessing and the dilemma of representation of cell decay. Another key innovation of CAP is the introduction of **adaptive event-guided** (abbreviated as **AEG**) sampling method and **rolling-as-window** (abbreviated as **RAW**) inference approach. Specifically, the AEG sampling is a temporal data sampling strategy for cell microscopy imaging sequence data to remedy the imbalance of cell division occurrences in the dataset; RAW is an inference approach to capture and track new cells within a long sequence in an efficient way. Our contributions are summarized as follows:

- We propose a novel **CAP** cell tracking framework that is highly efficient in terms of **inference time** and **data annotation requirements**, enabling **one-stage**

tracking that bypasses detection / segmentation to investigate cellular behaviors.

- We address prevalent challenges in the cell sequence data by proposing **AEG** sampling to resolve *imbalanced cell division occurrence, representing complex cell events* for training purposes through **cell point trajectory** and **visibility** concepts within a one-stage framework.
- We introduce a **RAW** inference approach in the CAP framework to *accurately capture and track new cells in long sequences* during frame sequences.
- Our proposed one-stage approach, CAP, achieves SOTA performance and is 8 to 32 times more efficient for inference than existing multi-stage methods, while alleviating the need for detection or segmentation.

2. Related Work

2.1. Cell Tracking

Cell tracking is a crucial task in biological imaging that typically involves the identification and temporal association of cell instances across image frames. Traditionally, this process includes two main steps: cell segmentation (detection) [14] and cell linking [15]. The segmentation step identifies individual cells in each frame, while the linking step establishes temporal associations between these cells across frames, sometimes capturing complex dynamics such as cell division and migration [8, 10]. Nowadays, instead of traditional approaches such as Nearest Neighbors linking [16], Bayesian filters [17, 18], Viterbi [19, 20], and graph-based matching [21, 22, 23], the advent of deep learning has transformed the landscape of cell tracking, leveraging advanced neural architectures for segmentation and tracking tasks. Deep learning approaches often employ complex models, e.g., CNN, RNN, GNN, and transformer, to combine multiple stages like detection [24, 25, 26, 27, 11, 12, 28], segmentation [29, 30, 8, 31, 32, 33, 6, 34], and tracking into the frameworks. Contemporary methods [35, 36, 25] aim to integrate contextual cell data into tracking models. ConvGRU [36] is introduced by Payer et al., capturing local cellular and inter-frame features. However, this method requires extensive annotated training data for all cell

regions, making it labor-intensive. Motion and Position Map (MPM) is capable of representing detection and association at once, which is proposed by Hayashida et al. [11], marking an advancement in simplifying the pipeline for cell tracking. However, it still requires multiple stages, first generating the MPM followed by subsequent detection and tracking procedures.

Thus, although existing multi-stage methods, which are based on individual cell detection followed by linking / association or evolving segmentation masks over time, show great cell tracking performance, they heavily rely on segmentation as a preliminary step before tracking. These methods often require high-quality data annotation for training and can be computationally intensive, especially when dealing with high-resolution data [37]. Additionally, these methods often involve complex pipelines that may not support end-to-end or one-stage training, posing challenges for both training and inference phases in terms of efficiency and scalability [13]. Thus, to tackle these issues, an end-to-end one-stage framework that omits the intermediate segmentation or detection stage is essential.

2.2. Point Tracking

In the field of natural images, recent end-to-end video point-tracking frameworks have achieved commendable performance. Notably, TAP-Vid [38] pioneered tracking any point across video frames, establishing a benchmark, and claiming a baseline method. However, this method does not support tracking occluded points, limiting its applicability in cell-tracking scenarios. Even though PIPs [39] and PointOdyssey [40] methods take measures to address the issue of point occlusion in long sequences by tracking selected points using a fixed sliding window and resuming tracking from the last visible frame, these methods struggle with point tracking independently, which may not capture the dependencies between multiple points. OmniMotion [41] accounts for estimated correspondences within a canonical space by optimizing a volumetric representation for video sequences. Although innovative, this method involves test-time optimization, making it computationally expensive and less viable for clinical applications. CoTracker [42] represents significant advancements in the field of point tracking. It addresses the inaccuracies led by independent tracking among points by

incorporating incremental point coordinates updates based on 4D correlation volumes, proposed by RAFT [43] constructed for all pairs of points from point/object features. CoTracker3 [44] is advanced by involving a semi-supervised training methodology to reduce reliance on synthetic datasets and utilizing pseudo-labeling techniques to generate training labels from unannotated real-world videos.

However, the trackers designed for general-purpose applications are ineffective for cell tracking because the unique properties of cell behaviors, i.e., cell mitosis and apoptosis, have not been considered in the model construction. Specifically, cells are not only relative with position and occlusion but also potentially associated with their ancestors due to the regulation of cellular life decay and division. Therefore, they cannot simultaneously represent or predict both cell positions and division states with their ancestors or children. Moreover, regarding the publicly available cell tracking data [7, 10, 45, 46, 47], the diversity of cell types results in significant variations in cell division rates and sequence length across different datasets categorized by specimens. Imbalanced cell event occurrence [48] in a sequence can lead to the omission of complete division events (from the death of the mother cell to the birth of the daughter cell) during random sampling with a low division rate; the inference for long sequences typically requires more time or computational resources to predict cell locations and lineages at once. Thus, existing methods potentially result in the *under-learning of division cases* in the model training process and a higher resource requirement in the *long-sequence inference* procedure. Thus, the uniqueness of this time-lapse cell sequential image data poses the most significant challenge in developing a one-stage cell tracking framework to effectively track cells jointly.

3. Methodology

3.1. Preliminaries

Recurrent All-Pairs Field Transforms (RAFT) [43] embedded in CoTracker [42] can be conceptualized as incrementally computing the similarity between all pairs of pixel points across frames in the sequence, enabling the model to handle large displacements and complex motion patterns. In the field of cell tracking, this design promotes

the cell point’s trajectories in a frame sequence constructed by iteratively updating the flow field using a recurrent unit that performs lookups on a 4D correlation volume.

4D Correlation Volume. Given two feature maps $F_1 \in \mathbb{R}^{H \times W \times dim}$ and $F_2 \in \mathbb{R}^{H \times W \times dim}$ with output dim -dimensions, which are features extracted from two input images I_1 and I_2 , by the feature extractor g_ϕ : $F_1 = g_\phi(I_1)$ and $F_2 = g_\phi(I_2)$. RAFT computes a correlation volume C that captures the similarity between all pairs of pixels across the two images. The correlation between a pixel at position (i, j) in F_1 and a pixel at position (k, l) in F_2 is calculated as the dot product of their feature vectors:

$$C_{ijkl} = \langle F_1(i, j), F_2(k, l) \rangle = \sum_h F_1(i, j, h) \cdot F_2(k, l, h), \quad (1)$$

which is a 4D tensor of dimensions $H \times W \times H \times W$, where H and W are respectively the height and width of feature maps F_s . To capture multi-scale feature information for large displacement, RAFT constructs a correlation pyramid by applying average pooling to the last two dimensions of the 4D correlation volume. Specifically, this pooling is performed with kernel sizes k of 1, 2, 4, and 8, and a set of multi-scale correlation volumes $C^k = \{C^1, C^2, C^3, C^4\}$ is generated. Thus, each volume C^k in the pyramid has dimensions $H \times W \times \frac{H}{2^k} \times \frac{W}{2^k}$.

Iterative Updates. The update operator estimates a sequence of flow estimations $\{f_1, \dots, f_n\}$ with initializing the start point $f_0 = 0$. For each iteration, it produces Δf as the direction that should be applied to the current estimation: $f_{k+1} = \Delta f + f_k$. Specifically, given the current flow estimation f_k , the correlation volume pyramid is retrieved and undergoes processing through two convolutional layers to extract relevant information. The updated information is comprised of correlation, flow, and context features and is updated over time as a feature map concatenation.

3.2. Proposed Method: Cell as Point (CAP)

Cell tracking aims to identify the characteristics of cell migration and division, thereby fully automating the prediction of specific cell position and division status changes in each frame of a time-lapse microscopy video. In our framework, CAP, every single cell is a point, tracking all cell points throughout the duration of a frame

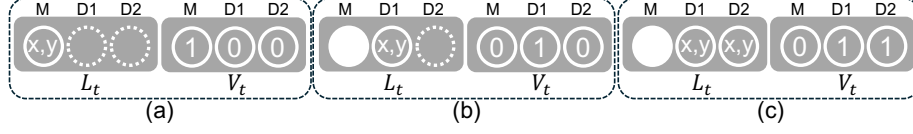


Figure 2: **Cell Point Trajectory and Visibility.** M, D1, and D2 denote the mother cell, the daughter cell #1, and daughter cell #2, respectively. (x,y) in L_t represent the location coordinates, and 0 or 1 in V_t respectively represent non-existing and existing cells. Three valid trajectory and visibility status: (a) M has not divided; (b) M has divided, and only D1 occurs in the t -th frame; (c) M has divided, and both D1 and D2 occur in the t -th frame.

sequence $S = (I_t)_{t=1}^T$ containing T grayscale cell microscopy images $I_t \in \mathbb{R}^{H \times W}$. *Cell point trajectories* and *visibilities* are proposed to be utilized in the CAP framework to represent and predict cell locations and lineages. The *cell point trajectory* is constituted by the N cell centroid point locations $L_t^i = (x_t^i, y_t^i) \in \mathbb{R}^2$, $i = 1, \dots, N$, $t = 1, \dots, T$, where t is the time of a frame sequence. The *cell point visibility* of the cell $V_t^i \in \{0, 1\}$ indicates whether the i -th cell point still exists in the frame sequence at time t ; in other words, if the cell is non-existing or born due to mitosis and apoptosis. Since any cell division that happened in the event would have been that of the parent cell into two daughter cells [49], Figure 2 demonstrates the combinations of cell point trajectory and visibility, representing various states of cells, i.e., migration, mitosis, apoptosis, and cell lineages (the mother cell with two potential daughter cells). This simple yet effective representation method will efficiently aid the CAP framework in achieving one-stage efficient cell tracking.

3.2.1. Adaptive Event-Guided (AEG) Sampling

To reduce operational costs and compromise overfitting [50], instead of seeing the entire training sequence in a single instance, the model learns a continuous sub-sequence of T_s frames for each time, where $0 < T_s < T$. However, the imbalanced occurrence of cell division in the entire sequence would cause the model to detect cell division occurring rarely. To facilitate the effective training of models to understand the biological behavior of the completed cell division event while mitigating data imbalance [51, 52], we propose the Adaptive Event-Guided (AEG) sampling technique to balance the occurrences of cell division in a sequence of frames the model sees by

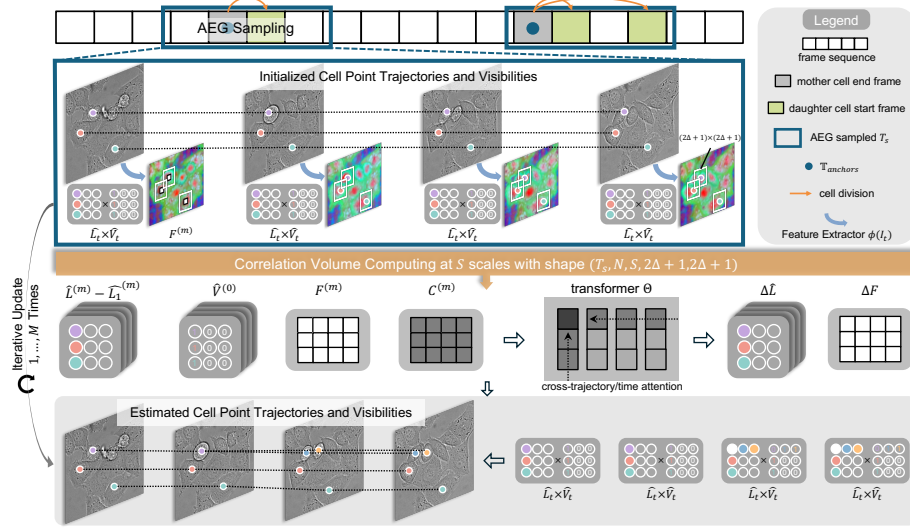


Figure 3: **Overview of CAP framework.** The training sequence T_s is sampled from the entire frame sequence using the AEG strategy. For each T_s , the cell point trajectories and visibilities are iteratively refined, starting from their initialization. For iteration $1, \dots, M$, updates are made to *cell point trajectory* $\hat{L}$ and *tracking feature* F with *cell point visibility* $\hat{V}$ computed at the final (M -th) update. This figure illustrates a single iteration of the optimization process.

determining the initial frame of T_s learning a complete event.

The AEG method leverages the intrinsic dynamics of cell division within the frame sequences to guide the sampling process, enhancing the representational diversity of the training dataset and ensuring the model’s robustness to variations in cell behavior. Technically, as data loading iterates through the whole dataset by detecting the complete cell division processes (from the death of the parent cell to the birth of the daughter cell), an anchor is added at each time cell division occurs $\mathbb{T}_{anchors}$. The probability of applying AEG P_{AEG} is adaptively calculated by $P_{AEG} = \frac{N_{div} \times T_{div}}{T}$, where N_{div} denotes the total occurrence of cell division in the entire T -frame sequence, and T_{div} denotes the duration of the completed cell division. When the random decimal $p < P_{AEG}$ indicates the usage of AEG, our CAP framework incorporates AEG sampling for selecting T_s , as illustrated at the top of Figure 3 (dark blue square); otherwise, the starting frame of T_s is randomly located in the $(0, T - T_s)$ range.

For each training sequence, we crop a fixed-length frame sequence in accordance

with the starting frame from the video. Occasionally, the process of cell division, resulting in the formation of new cells, may span multiple frames. Thus, the leverage of the AEG approach allows us to capture complete division processes within sampled sequences in data pre-processing. This fixed-length cropping is critical to maintaining uniformity in the input data, facilitating more stable and reliable learning outcomes.

3.2.2. Cell Joint Tracking

By regarding each cell as a point and leveraging the two-dimensional feature across time and a set of tracked cell points, we implement an effective method for cell tracking via the correlation between cell point trajectories and visibilities in one stage. The training process of CAP as a transformer Θ neural network, where $\Theta : G \mapsto O$. In order to improve the estimated cell trajectories and visibilities along with M iterations, the trajectories and visibilities of i points (cells) in a t -frame sequence are encoded into a grid of input tokens G_t^i . O_t^i is the corresponding output of Θ for representing the refined trajectories and visibilities. The following sections elaborate on the transformer formulation visualized in Figure 3.

Feature Extraction. To initialize the tracking feature F for the part of input tokens G , we first generate **image features** (feature map), $\phi(I_t)$, for each frame in the training sequence, from which the **tracking features**, F , are distilled according to cell point locations. Specifically, the feature extractor is a convolutional neural network (CNN) trained end-to-end in the framework for computational efficiency. Leveraging the CNN feature extractor, the feature maps with dim dimensions are generated from I_t , $\phi(I_t) \in \mathbb{R}^{dim \times (\lfloor \frac{H}{k} \rfloor \times \lfloor \frac{W}{k} \rfloor)}$, where $k = 4$ for computational efficiency, and the features are downsampled by average pooling with strides s for multi-scaled features. The tracking features $F_{t=1}^i$ for i are captured by bilinear interpolation from the location of cell points on the initial frame $L_{t=1}$ on the feature map $\phi(I_{t=1})$. The tracking feature vector $F_t^i \in \mathbb{R}^{dim}$ is initialized for the transformer input by broadcasting the tracking features in $t = 1$ frame for all T_s frames, which is refined and updated with M iterations.

Correlation Volume. To achieve the objective of associating cell point trajectories jointly, CAP employs correlation volume $C_t^i \in \mathbb{R}^S$, where $S = 4$, proposed in RAFT [43].

These correlation volumes C_t^i are derived by assessing the relationship between the tracking features F_t^i and the image features $\phi(I_t)$ in proximity to the estimated cell point location $\hat{L}_t^i$ of the trajectory on the t -th frame. Technically, each C_t^i is the average over the correlation response in a $2^s \times 2^s$ grid:

$$\begin{aligned} [C_t^i]_\delta^s &= \frac{1}{2^{2s}} \sum_{\delta} \sum_{\delta} \langle F_t^i, \phi_s(I_t)_{2^s \hat{x} + \delta, 2^s \hat{y} + \delta} \rangle \\ &= \langle F_t^i, \frac{1}{2^{2s}} (\sum_{\delta} \sum_{\delta} \phi_s(I_t)_{2^s \hat{x} + \delta, 2^s \hat{y} + \delta}) \rangle, \end{aligned} \quad (2)$$

where $s = 1, \dots, S$ are the feature scales, and δ is the offset around cell points, $(\hat{x}, \hat{y})$ is the estimated location coordinate of the cell point $\hat{L}_t^i$. The value of $[C_t^i]_\delta^s$ is computed as the inner product between the feature vector F_t^i and $\phi_s(I_t)$ at $\hat{L}_t^i$ pooled with kernel size $2^s \times 2^s$.

Iterative Updates. In the transformer-based interactive updates, the input tokens express the concatenation of features: location $\hat{L}_t^i - \hat{L}_1^i$, visibility $\hat{V}_t^i$, tracking features F_t^i , and correlation volume of cell trajectories C_t^i . As for the output tokens, O_t^i contains updates for location and refined tracking features: $(\Delta \hat{L}_t^{3i}, \Delta F_t^i)$. CAP estimates 3 locations for each cell, following the representation illustrated in Figure 2: the cell’s location and the locations of two potential daughter cells. In this way, CAP reserves space for each cell’s progeny by representing L and V through a 3-column format. M -times updates the output tokens by applying the transformer Θ iteratively for a sequence T_s as shown in Figure 3. Specifically, CAP utilizes transformer Θ with cross-trajectory/time attention to update both the cell point trajectory, $\hat{L}$, and the tracking feature, F :

$$\hat{L}^{(m+1)} = \hat{L}^{(m)} + \Delta \hat{L} \text{ and } F^{(m+1)} = F^{(m)} + \Delta F, \quad (3)$$

where $m = 1, \dots, M$, and $m = 1$ denotes initialization. At the last iteration of the update M , cell point visibility V_t^{3i} for locations estimated $\hat{L}_t^{3i}$ is computed by $\hat{V}^{(M)} = \sigma(WF^{(M)})$, where σ represents the sigmoid activation function and W is a learned matrix of weights. By investigating the 3 visibilities V_t^{3i} , CAP tells if the cell division occurred on the cell i in the t -th frame.

Loss Function. We supervised the transformer Θ on the *trajectory* and *visibility* between predicted and ground truth for every semi-overlapping window. The trajectory loss function $\mathcal{L}_{tra}$ is location regression summed over the iteration updates, and the visibility loss function $\mathcal{L}_{vis}$ is the cross-entropy of binary visibility options:

$$\mathcal{L}_{tra}(\hat{L}, L) = \sum_{j=1}^J \sum_{m=1}^M \gamma^{M-m} \|\hat{L}^{(m,j)} - L^{(j)}\|, \quad (4)$$

$$\mathcal{L}_{vis}(\hat{V}, V) = \sum_{j=1}^J \text{CE}(\hat{V}^{(M,j)}, V^{(j)}), \quad (5)$$

where $\mathcal{L}_{vis}$ is updated at the M -th iteration, and J is the total number of T_s whose trajectory starts in the middle of the previous T_s .

Algorithm 1: Algorithm of RAW Inference

Input: $\Theta, S_{\text{infer}} \leftarrow (I_t)_{t=1}^{T_{\text{infer}}}, Q \leftarrow (x^i, y^i), l_{\text{win}}$
Output: L_t, V_t , where $t \leftarrow 1, \dots, T_{\text{infer}}$
Initialization: current process frame $t_{\text{cur}} \leftarrow 1$
while $t_{\text{cur}} < T_{\text{infer}}$ **do**
 Obtain current window frames $S_w \leftarrow (I_t)_{t=t_{\text{cur}}}^{t_{\text{cur}}+l_{\text{win}}}$
 $L_w, V_w \leftarrow \Theta(S_w)$
 for $i \leftarrow 0$ **to** l_{win} **do**
 $t_{\text{cur}} \leftarrow t_{\text{cur}} + i$
 $Q \leftarrow L_{t_{\text{cur}}}$
 if find new cell(s) **then**
 Concatenate Q and $\hat{L}_{t_{\text{cur}}}^{\text{new}}$
 break
 end
 end
end

3.2.3. Rolling-as-Window (RAW) Inference

In cell tracking, the inference stage is crucial for producing the final tracking mask and establishing cell lineage relationships by analyzing predicted cell locations and division events. Existing works [8, 32, 53] often customize inference and post-processing

stages with different configurations for each dataset, adapting to variations in sequence length. This data-specific tailoring limits their generalizability and broader applicability to other datasets. In contrast, our proposed Rolling-As-Window (RAW) inference approach in CAP simplifies and unifies these processes while maintaining effectiveness across varying sequence lengths, enabling high-quality inference regardless of computational constraints. Specifically, a consistent and high-quality inference result is achieved through a rolling loop, which sequentially extracts yet-to-be-inferred clips from the full sequence, processing them within a defined window for inference.

In a certain sequence with T_{infer} frames, if the inference window length l_{win} supported by either computational resources or architecture is shorter than T_{infer} , the first rolling of window frames $S_w = (I_t)_{t=1}^{l_{\text{win}}}$ is passed into the trained transformer Θ ; otherwise, the whole sequence will be inferred at once. The RAW inference approach is summarized in Algorithm 1, which will be triggered while the former scenario happens. Specifically, a l_{win} -length subsequence starting from $t = 1$ is passed into Θ for prediction. The output, cell point trajectories L_t and visibilities V_t , where $t = 1, \dots, l_{\text{win}}$, are processed frame-by-frame to update query points Q based on the L_t of identified cells and check for cell division or the emergence of new cells. Whenever a new cell appears, the predicted location of this new cell $\hat{L}_t^{\text{new}}$ is incorporated into the updated query points Q for the subsequent rolling of analysis by the formula:

$$Q_{r+1} = \text{concat}(Q_r, \hat{L}_t^{\text{new}}) = \begin{bmatrix} Q_r \\ \hat{L}_t^{\text{new}} \end{bmatrix}, \quad (6)$$

where r is the times of rolling for window inference. Subsequently, RAW segments a new window sequence starting from the frame where the new cell appeared, preparing it for the next application of Θ . By adjusting the l_{win} based on the capacity of computational resources and T_{infer} value, our CAP is capable of performing efficient and high-quality inference on sequences of any length.

4. Experiments and Results

We conducted extensive experiments to demonstrate the high efficacy and efficiency of CAP, our end-to-end, one-stage framework for cell tracking. CAP maintains

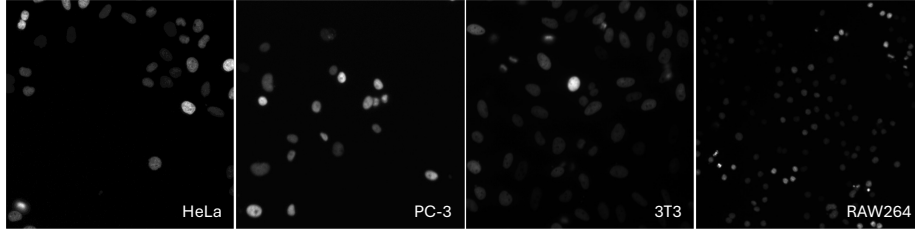


Figure 4: **Visualization of the DeepCell Database.** The datasets HeLa, PC-3, 3T3, and RAW264 have different cell densities, sizes, and luminances.

the promising performance, despite requiring minimal inference time and eliminating the need for additional annotations. Additionally, we perform comprehensive ablation studies to assess the effectiveness of individual CAP components.

4.1. Experimental Setup

4.1.1. Datasets

We evaluate our proposed framework on DynamicNuclearNet Tracking in DeepCell [7] public database, HeLa, PC-3, 3T3, and RAW264 datasets divided by specimen, and ISBI cell tracking challenge (ISBI CTC) [10, 45] public datasets, PhC-C2DH-U373 (U373), Fluo-N2DH-GOWT1 (GOWT1), and Fluo-C2DL-Huh7 (Huh7). These temporal data used for cell tracking, shot by different devices in various durations on diverse specimens, exhibit diverse morphologies and sequence characteristics.

DeepCell. DeepCell database consists of HeLa, PC-3, 3T3, and RAW264 datasets, which respectively contains cell types HeLa-S3, PC-3, NIH-3T3, and RAW 264.7, visualized in Figure 4. These four cell types are mammalian cell lines that were acquired from the American Type Culture Collection [7]. The cells are cultured in Dulbecco’s modified Eagle’s medium (RAW 264.7 and NIH-3T3) or F-12K medium (Hela-S3 and PC-3) supplemented with the necessary drugs, e.g., L-glutamine, penicillin, for all other cells. DeepCell contains 96 sequences in total: 45 for HeLa, 7 for PC-3, 10 for 3T3, and 34 for RAW264. Among them, there are 5, 2, 1, and 3 sequences for datasets HeLa, PC-3, 3T3, and RAW264 for evaluating the training outcomes of the remaining sequences, respectively. The detailed information on each cell type is illustrated below:

- *HeLa* includes 77,180 objects, 2,234 tracks, and 189 cell divisions, with a relatively high cell density. Each sequence is segmented into 42 frames, lasting 6 minutes, containing cell behaviors, i.e., cell migration and division. For each frame in the sequences, they are 540×540 with a pixel size 0.65.
- *PC-3* is the smallest subset in DynamicNuclearNet, with 5,051 objects, 159 tracks, and only 5 divisions, captured in a 5-minute duration. All sequences are segmented into 50 frames with a size 584×584 and pixel size 0.55.
- *3T3* cells are fibroblasts from mouse embryonic tissue. In this paper, this cell line contributes 62,983 objects, 1,413 tracks, and 250 divisions, offering insights into cell growth and cycle dynamics. There are 71 frames in each sequence, and each frame is 512×512 with a pixel size of 0.65.
- The *RAW264* cell line, derived from murine macrophages, includes 335,049 objects, 8,642 tracks, and 1,322 divisions, making it the largest subset in our study. Each sequence has 45 frames with a size of 540×540 with a pixel size of 0.65. The cell density in this dataset is the highest, with the smallest cell size.

ISBI CTC. Figure 5 illustrates datasets we leveraged, U373, GOWT1, and Huh7, with the comparison between tracking (TRA) GT mask, segmentation (SEG) GT, and segmentation (SEG) ST if available. We observe that SEG GT and ST are much more refined and high-quality than TRA GT masks, requiring more resources and manpower. In terms of data annotation requirements, unlike previous works that rely on all available TRA GT masks, SEG GT, and ST (when available), CAP requires only the TRA GT masks, thereby substantially reducing the annotation demand. The detailed information on data collection and the characteristics of each dataset is described below:

- The *PhC-C2DH-U373* dataset comprises active glioblastoma-astrocytoma U373 cells on a polyacrylamide substrate. Each sequence is captured by a Nikon microscope with an object lens Plan Fluor DLL 20x/0.5 for a relatively long duration of 15 minutes, totaling 115 frames with the size of 520×696 . The cells are relatively sparse in frames. The distinct feature of this dataset, compared to others, is its higher grayscale background.

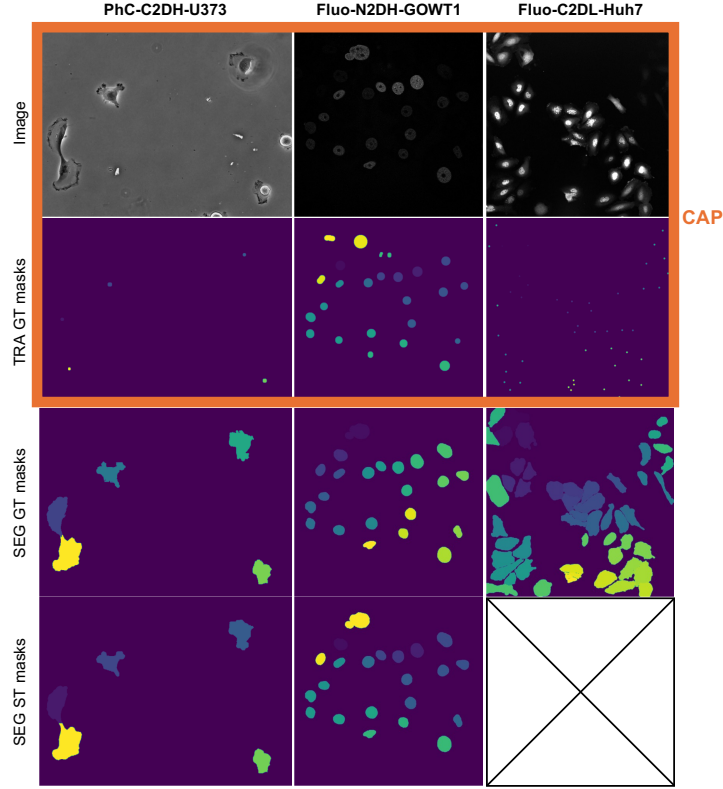


Figure 5: **Comparison of Quality and Requirement of Data Annotation.** Fluo-C2DL-Huh7 does NOT have segmentation ST. Instead of previous works utilizing all types of mask information with original images in the figure, CAP only utilizes the Tracking GT masks with original images (in the orange block).

- Dataset *Fluo-N2DH-GOWT1* documents the activity of GFP-GOWT1 mouse stem cells over a 5-minute period as observed by Leica TCS SP5 microscope with objective lens Plan-Apochromat 63x/1.4 (oil), segmented into 92 frames with a size of 1024×1024 per sequence. The target cells for tracking are notably active in cell events and exhibit a relatively high density within the frames.
- *Fluo-C2DL-Huh7* is a dataset of human hepatocarcinoma-derived cells expressing the fusion protein YFP-TIA-1. Cell events are captured by the Nikon Eclipse Ti2 microscope with a lens CFI Plan Apo Lambda 20x/0.75 in a duration of 15 minutes with 30 frames for each sequence. Cells in this dataset exhibit a rela-

Layer / Block	Kernel	#Layers / Blocks	Normalization
Conv2D	7×7	1	–
Residual blocks	3×3	8	InstanceNorm
Conv2D	3×3	1	–
Conv2D	1×1	1	–

Table 1: **Architecture Details of the 2D CNN Feature Extractor.**

tively low frequency of cell events; However, due to the lower frame rate, cells exhibit significantly larger movements within a sequence. Cells are relatively small and have high density in frames.

4.1.2. Implementation Details

For the **training** process, we utilize 85 training sequences from the DeepCell database and sequences 02 from the ISBI dataset, using only tracking (TRA) ground truth (GT). During each iteration, we crop sequences of $T_s = 24$ frames with a sliding window length of $l_{\text{slide}} = 8$. We use pre-trained checkpoints from CoTracker [42], which was trained solely on synthetic RGB data. Since our input images are grayscale, we replicate the single channel three times to match the required RGB format. We use an end-to-end trained 2-dimensional CNN architecture as the encoder to extract tracking features F . Specifically, it integrates bilinear interpolation and utilizes it to extract $S = 4$ different scaled image features with 128-channel output features concatenated to optimized dynamic feature extraction. The architecture details of the 2D CNN feature extractor are listed in Table 1. All the experiments are trained and tested on an NVIDIA GeForce RTX 4090 (24GB) GPU. The total end-to-end training time takes approximately 38 minutes for the DeepCell database and 20 minutes for the CTC datasets. The entire CAP framework contains a total of 194.8M learnable parameters. While training the transformer, we follow the pre-trained checkpoint that downsamples the resolution of frames to 384×512 and choose $S = 4$, $\Delta = 3$ to calculate 4D multi-scaled correlation volumes. As for the optimization during the training, we present an optimizer configuration in Table 2.

As for the **testing (evaluation)** process, we preprocess the first frame for each

Hyperparameter	Value
Batch Size	2
Number of steps for DeepCell database	2500 (approx. 38mins)
Number of steps for CTC datasets	700 (approx. 20mins)
Learning rate scheduler	OneCycleLR
Upper learning rate	5e-4
Weight decay	1e-5
Epsilon	1e-8
Optimizer	AdamW
Percentage of the cycle increasing the learning rate	0.05
Cycle momentum	False
Annealing strategy	linear
Value of the discount factor γ for $\mathcal{L}_{tra}$	0.8

Table 2: **Optimizer Configuration for CAP.**

sequence using [33] to select query points $\mathcal{Q}$ and start the process of window rolling. The evaluation sequence S_{infer} follows the native resolution (384×512), which the dataset is trained on. We set $l_{\text{win}} = 50$ for DeepCell database and $l_{\text{win}} = 100$ for CTC datasets, which is mentioned in Sec. 3.2.3. We follow the TRA evaluation metrics [54, 55] of the operations provided by ISBI [10, 45] and CTMC [47] to correct the situations of merged cells, missing links, wrong links, links with wrong semantics, false positives, and false negatives: EdgeSplit (ES), EdgeAdd (EA), EdgeDelete (ED), EdgeSemantic (ESM), FP, and FN. TRA lies in the range $[0, 1]$ where a higher score corresponds with a better performance, and is calculated by:

$$\text{TRA} = 1 - \frac{\min(\text{AOGM}, \text{AOGM}_0)}{\text{AOGM}_0}, \text{ where} \quad (7)$$

$$\text{AOGM} = w_{\text{ES}}\text{ES} + w_{\text{EA}}\text{EA} + w_{\text{ED}}\text{ED} + w_{\text{ESM}}\text{ESM} + w_{\text{FP}}\text{FP} + w_{\text{FN}}\text{FN}. \quad (8)$$

Here AOGM is the weighted sum of the executed operations considered as the cost of transforming (ES, EA, ED, ESM, FP, and FN) the computed graph into the reference one, and AOGM_0 is the AOGM value for empty tracking results. The details of ES,

EA, ED, ESM, FP, and FN are elaborated as follows¹:

ES is the operation for non-split vertices that the computed markers with more than one reference marker assigned;

EA is the operation for missing links that the cell lineage relations are not indicated;

ED is the operation for redundant links that the fake cell lineage relations are indicated;

ESM is the operation for altering the edge semantics, that is, the difference in semantics between the reference graph and the induced subgraph;

FP is the situation in the extra detected objects;

FN is the situation in the missed objects.

Baselines. To evaluate the performance of CAP, we conducted a comparison experiment with baseline methods, cell-tracker-gnn (GNN) [32], EmbedTrack [8], KIT-GE [33], Trackastra [6], Cell-TRACTR [34], and EBB [28], which have publicly available code and utilize the evaluation system provided by ISBI [10, 45] and the acyclic oriented graphs matching (AOGM) measure [54]. For the DeepCell data, we follow the experiments [7] and the default training/testing subsets to report the result. For the ISBI CTC dataset, to control for variables and ensure fairness, we train these methods on sequence 02 and evaluate on 01, following [33]. Since the EBB baseline method does not provide pretrained weights for detection (U373 and GOWT1), we re-run its provided code under the same experimental setup for training, test by the same metrics, and report the results. We report the measures of metrics involved in the cell tracking, EdgeSplit, EdgeAdd, EdgeDelete, EdgeSemantic, FP, FN, TRA, and the average inference time (IT) to evaluate the cell tracking performance and the inference efficiency.

4.2. Results and Analysis

We performed comparison experiments of our method, CAP, against baseline approaches, including GNN, EmbedTrack, KIT-GE, Trackastra, Cell-TRACTR, and EBB across the DeepCell database (HeLa, PC-3, 3T3, and RAW264) and the ISBI datasets (U373, GOWT1, and Huh7). The evaluation is based on both cell-tracking accuracy-

¹<https://celltrackingchallenge.net/evaluation-methodology/>

related metrics (TRA derived from EdgeSplit, EdgeAdd, EdgeDelete, EdgeSemantic, FP, and FN) and inference efficiency (IT).

Table 3 demonstrates the quantitative comparison results on DeepCell database. On HeLa and PC-3, CAP achieves the highest TRA scores of 0.926 and 0.952, surpassing prior works while requiring only 1.3s and 1.1s inference time, respectively. This highlights CAP’s ability to deliver state-of-the-art accuracy with minimal computational cost. On RAW264, CAP attains a competitive TRA of 0.864 with just 2.8s inference, demonstrating a 3 – 20 $\times$ reduction in runtime compared to other methods. Even on the challenging 3T3 dataset, CAP achieves a strong TRA of 0.854, comparable to the best baseline (0.857), but with an inference time of only 2.2s versus up to 65.0s for competing methods. As for the ISBI CTC datasets, as shown in Table 4, on U373, CAP achieves the best overall performance, recording zero errors across tracking structural metrics and achieving a TRA of 0.985, with the lowest inference time of 2.6s. For GOWT1, CAP achieves a TRA of 0.960, slightly below KIT-GE (0.966), while reducing inference time to 7.1s, corresponding to a 24 $\times$ speedup. On Huh7, CAP matches the best reported TRA (0.966) with the shortest inference time of 3.2s.

Beyond the aggregate TRA score, we further interpret CAP’s performance on cell division/lineage reconstruction using the structural error decomposition that underlies TRA (EdgeSplit, EdgeAdd, EdgeDelete, and EdgeSemantic). On DeepCell database, CAP yields zero EdgeSemantic errors on HeLa and PC-3 while maintaining strong TRA, indicating accurate mother–daughter relation recognition rather than only improved runtime. This advantage is especially meaningful on division-scarce PC-3 (only 5 divisions), where CAP reduces EdgeAdd to 11 (vs. 26~233 for other methods) while keeping EdgeDelete/EdgeSemantic at 0, which is consistent with the goal of mitigating division imbalance via AEG sampling. On division-rich RAW264 (1322 divisions), CAP remains competitive in lineage-structure errors (e.g., EdgeDelete = 12 and EdgeSemantic = 17) while achieving substantially the fastest inference, suggesting robust division tracking under frequent mitosis and crowded conditions. Similar trends are observed on the ISBI CTC datasets: CAP achieves all zero structural errors on U373 and maintains low EdgeSemantic/EdgeDelete on GOWT1 and Huh7. Qualitative lineage examples in Figures 7 and 8 further illustrate CAP’s ability to correctly

Method	Type	HeLa							
		EdgeSplit ↓	EdgeAdd ↓	EdgeDelete ↓	EdgeSemantic ↓	FP ↓	FN ↓	TRA ↑	IT ↓
GNN [32] 22'	SEG+TRA	2	385	1	13	62	342	0.776	23.3
EmbedTrack [8] 22'	SEG+TRA	0	211	1	1	31	218	0.849	41.5
KIT-GE [33] 20'	SEG+TRA	6	57	1	17	70	67	<u>0.923</u>	14.4
Trackastra [6] 24'	SEG+TRA	1	240	1	1	124	185	0.825	<u>11.8</u>
Cell-TRACTR [34] 25'	SEG+TRA	6	209	1	1	715	144	0.816	30.3
CAP (Ours)	TRA	3	82	1	0	60	70	0.926	1.3

Method	Type	PC-3							
		EdgeSplit ↓	EdgeAdd ↓	EdgeDelete ↓	EdgeSemantic ↓	FP ↓	FN ↓	TRA ↑	IT ↓
GNN [32] 22'	SEG+TRA	2	52	3	12	33	0	0.868	18.3
EmbedTrack [8] 22'	SEG+TRA	0	43	3	1	0	42	0.912	48.7
KIT-GE [33] 20'	SEG+TRA	29	56	0	0	13	1	<u>0.937</u>	9.1
Trackastra [6] 24'	SEG+TRA	2	233	2	1	155	190	0.317	<u>8.9</u>
Cell-TRACTR [34] 25'	SEG+TRA	1	26	1	0	146	8	0.913	33.7
CAP (Ours)	TRA	1	11	0	0	35	7	0.952	1.1

Method	Type	3T3							
		EdgeSplit ↓	EdgeAdd ↓	EdgeDelete ↓	EdgeSemantic ↓	FP ↓	FN ↓	TRA ↑	IT ↓
GNN [32] 22'	SEG+TRA	6	613	16	3	100	9	0.857	21.7
EmbedTrack [8] 22'	SEG+TRA	2	603	14	4	201	589	0.853	74.6
KIT-GE [33] 20'	SEG+TRA	128	783	14	10	256	698	0.820	19.1
Trackastra [6] 24'	SEG+TRA	10	2597	16	21	1511	1936	0.572	<u>12.0</u>
Cell-TRACTR [34] 25'	SEG+TRA	6	757	3	10	11834	213	0.727	65.0
CAP (Ours)	TRA	53	849	2	4	189	509	<u>0.854</u>	2.2

Method	Type	RAW264							
		EdgeSplit ↓	EdgeAdd ↓	EdgeDelete ↓	EdgeSemantic ↓	FP ↓	FN ↓	TRA ↑	IT ↓
GNN [32] 22'	SEG+TRA	186	1451	89	21	239	1341	0.801	<u>28.5</u>
EmbedTrack [8] 22'	SEG+TRA	68	1475	74	28	669	1259	0.803	59.6
KIT-GE [33] 20'	SEG+TRA	37	585	106	24	718	603	0.876	96.7
Trackastra [6] 24'	SEG+TRA	14	4572	82	46	2810	3354	0.490	56.3
Cell-TRACTR [34] 25'	SEG+TRA	21	1346	11	12	16617	354	0.713	51.5
CAP (Ours)	TRA	67	1098	12	17	647	898	<u>0.864</u>	2.8

Table 3: **Comparison Experiments of DeepCell Database.** We compare CAP to existing cell tracking frameworks available on the HeLa, PC-3, 3T3, and RAW264 datasets in the DeepCell database. The method *Type* is defined as follows: the method following SEG+TRA (segmentation + tracking style) and TRA (tracking-only (no explicit segmentation) style).

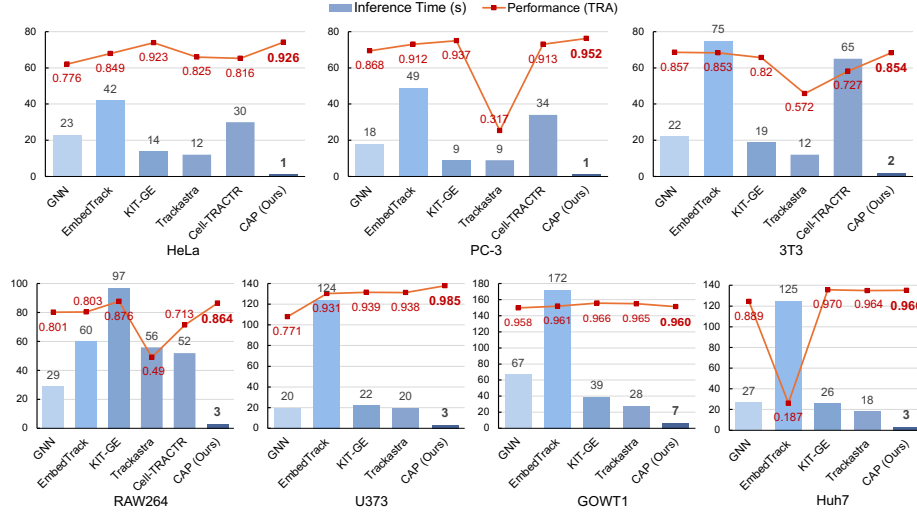


Figure 6: **Result of Tracking Accuracy and Inference Efficiency across Multiple Benchmark Datasets.** CAP consistently achieves the best trade-off between accuracy and efficiency, delivering competitive or superior performance while reducing inference time.

localize division points and preserve mother-daughter continuity even with simultaneous divisions.

As for the extreme instability of the EmbedTrack and EBB methods on Huh7 with a TRA of 0.187 and 0.626, respectively, our analysis suggests that the observed results stem from the method’s excessive reliance on the segmentation (SEG) stage and the usage of refined masks during the training of the segmentation model, while Huh7 contains only SEG golden truth (GT) masks without the more refined SEG silver truth (ST). This further substantiates an end-to-end one-stage cell tracking framework, which bypasses the segmentation or detection stage, enhances model stability, and raises the overall performance limits achievable by the model.

Overall, as shown in Figure 6, the CAP method demonstrates promising performance and constant high efficiency on inference and label usage across all datasets, consistently achieving high TRA scores and the lowest ITs. Thus, CAP is a highly effective, efficient, and reliable method for cell tracking in a practical biomedical field.

Method	Type	U373							
		EdgeSplit ↓	EdgeAdd ↓	EdgeDelete ↓	EdgeSemantic ↓	FP ↓	FN ↓	TRA ↑	IT ↓
GNN [32] 22'	SEG+TRA	214	371	0	0	135	25	0.771	<u>19.7</u>
EmbedTrack [8] 22'	SEG+TRA	0	44	0	7	211	32	0.931	123.6
KIT-GE [33] 20'	SEG+TRA	3	50	0	1	197	25	0.939	22.0
Trackastra [6] 24'	SEG+TRA	3	50	0	21	182	25	0.938	20.3
EBB [28] 25'	DET+TRA	2	37	0	22	143	22	<u>0.959</u>	45.6
CAP (Ours)	TRA	0	0	0	0	119	0	0.985	2.6

Method	Type	GOWT1							
		EdgeSplit ↓	EdgeAdd ↓	EdgeDelete ↓	EdgeSemantic ↓	FP ↓	FN ↓	TRA ↑	IT ↓
GNN [32] 22'	SEG+TRA	49	140	1	13	5	28	0.958	67.2
EmbedTrack [8] 22'	SEG+TRA	39	119	0	0	21	40	0.961	171.6
KIT-GE [33] 20'	SEG+TRA	45	111	0	1	6	32	<u>0.966</u>	39.4
Trackastra [6] 24'	SEG+TRA	47	140	0	1	4	36	0.965	<u>27.6</u>
EBB [28] 25'	DET+TRA	45	129	0	0	5	34	0.980	50.3
CAP (Ours)	TRA	18	120	2	2	75	46	0.960	7.1

Method	Type	Huh7							
		EdgeSplit ↓	EdgeAdd ↓	EdgeDelete ↓	EdgeSemantic ↓	FP ↓	FN ↓	TRA ↑	IT ↓
GNN [32] 22'	SEG+TRA	34	85	2	835	36	4	0.889	27.1
EmbedTrack [8] 22'	SEG+TRA	0	733	0	5	1392	636	0.187	125.0
KIT-GE [33] 20'	SEG+TRA	34	77	0	0	41	0	0.970	25.7
Trackastra [6] 24'	SEG+TRA	34	84	2	5	34	5	0.964	<u>18.2</u>
EBB [28] 25'	DET+TRA	27	588	0	4	870	247	0.620	19.0
CAP (Ours)	TRA	31	71	0	1	101	1	<u>0.966</u>	3.2

Table 4: **Comparison Experiments of the ISBI CTC Dataset.** We compare CAP to existing cell tracking frameworks available on the datasets PhC-C2DH-U373, Fluo-N2DH-GOWT1, and Fluo-C2DL-Huh7. The method *Type* is defined as follows: the method following SEG/DET+TRA (segmentation/detection + tracking style) and TRA (tracking-only (no explicit segmentation/detection stage) style).

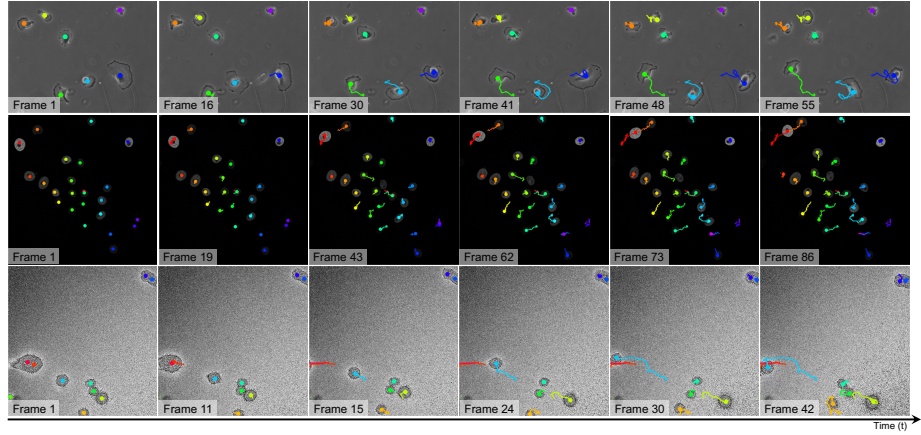


Figure 7: **Cell Trajectories.** Each point represents a cell with distinct colors, indicating individual cell trajectories across time points. The cell density for each trajectory follows the order: medium, high, and low.

4.3. Qualitative Result

To further validate the performance of CAP, we provide qualitative results illustrating two core aspects of cell tracking: cell trajectory estimation and lineage (division) reconstruction.

Figure 7 visualizes representative examples of cell trajectories across multiple time points. Each cell is assigned a distinct color, enabling clear visualization of its movement path. The results demonstrate CAP’s ability to robustly maintain cell identities over extended sequences, even in the presence of densely packed regions, complex motion patterns, and various motion speeds. This consistency highlights CAP’s strength in long-term trajectory preservation and cross-frame association accuracy.

Figure 8 illustrates the model’s performance on capturing cell division events. CAP successfully identifies division points and accurately tracks the transition from mother cells (M) to daughter cells (D1 and D2), ensuring continuity across frames. The highlighted examples show that CAP not only maintains the mother–daughter relationships but also effectively disambiguates multiple simultaneous divisions within crowded environments. These results underscore the model’s capacity to reconstruct biologically meaningful lineage information, which is critical for downstream analysis in developmental biology and cancer research.

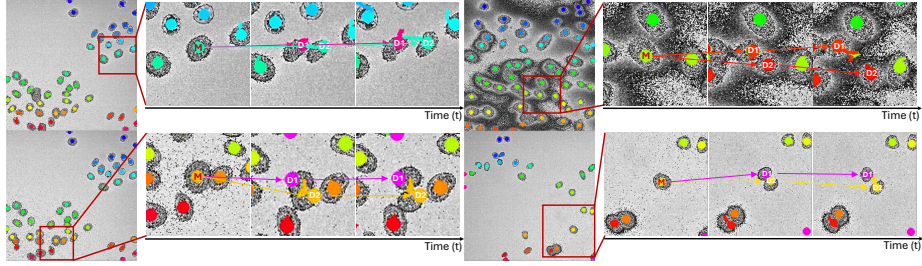


Figure 8: **Cell Lineage (Divisions)**. Utilizing the proposed Cell Point Trajectory and Visibility representation illustrated in Figure 2, CAP is able to correctly identify the cell division and track all cells in the sequence. **M** denotes the mother cell; **D1** and **D2** represent the daughter cell #1 and the daughter cell #2.

AEG Sampling	U373	GOWT1	Huh7	HeLa					PC-3		3T3	RAW264		
				01	02	03	04	05	01	02	01	01	02	03
$\times$	0.556	0.941	0.942	0.898	0.908	0.883	0.921	0.878	0.952	0.924	0.829	0.804	0.828	0.843
$\checkmark$	0.985	0.960	0.966	0.962	0.958	0.921	0.900	0.891	0.980	0.924	0.832	0.824	0.868	0.900

Table 5: **Effectiveness of AEG Sampling**. AEG sampling is built to balance the cell event (division) in a training sequence.

4.4. Ablation Study

We performed extensive experiments to assess the effectiveness of each key component in the CAP framework on both the DeepCell and ISBI CTC datasets: the AEG sampling strategy, the cell joint tracking structure, and the feature stride of the model. To more clearly observe the influence of each key component on individual test sequences, we report the TRA results for each sequence in the DeepCell database.

Effectiveness of AEG Sampling. To address the imbalance of cell division events during training, we propose the Adaptive Event-Guided (AEG) sampling strategy (Section 3.2.1), which calibrates the training process by ensuring balanced exposure to complete division sequences. As shown in Table 5, disabling AEG sampling leads to substantial performance degradation across datasets. With AEG sampling enabled, the model achieves consistent improvements: for instance, on the HeLa dataset, the best TRA score rises from 0.898 to 0.962, while on U373 it improves from 0.556 to 0.985. Similar gains are observed on PC-3 and RAW264. These results confirm that

Attention					HeLa					PC-3		3T3	RAW264		
time	joint	U373	GOWT1	Huh7	01	02	03	04	05	01	02	01	01	02	03
✓	✗	0.974	0.903	0.944	0.853	0.949	0.861	0.921	0.906	0.925	0.947	0.812	0.805	0.845	0.821
✓	✓	0.985	0.960	0.966	0.962	0.958	0.921	0.900	0.891	0.980	0.924	0.832	0.824	0.868	0.900

Table 6: **Effectiveness of Cell Joint Tracking.** We compare CAP performance using time and cross-trajectory attention.

Stride				HeLa					PC-3		3T3	RAW264		
	U373	GOWT1	Huh7	01	02	03	04	05	01	02	01	01	02	03
8	0.531	0.609	0.853	0.884	0.919	0.806	0.936	0.853	0.105	0.274	0.387	0.610	0.622	0.758
4	0.985	0.960	0.966	0.962	0.958	0.921	0.900	0.891	0.980	0.924	0.832	0.824	0.868	0.900

Table 7: **Ablation on Feature Stride.** CAP predicts more accurately with higher resolution features.

AEG sampling effectively balances event occurrences and significantly enhances the model’s ability to learn from imbalanced data.

Effectiveness of Cell Joint Tracking. We evaluate the role of joint tracking by replacing cross-trajectory attention layers with time-only attention while maintaining model size. As reported in Table 6, this modification consistently reduces performance, with TRA drops ranging from 0.015 to 0.055 across datasets. Incorporating both time and joint attention yields notable improvements, particularly on HeLa where the TRA increases from 0.853 to 0.962 in the best case. This demonstrates that the dual-attention design not only strengthens temporal coherence but also captures relational dependencies between trajectories, leading to more robust tracking in complex cellular environments.

Ablation on Feature Stride. Table 7 highlights the effect of feature stride on model accuracy. Reducing the stride from 8 to 4 substantially improves performance by providing higher-resolution features. For example, on the HeLa dataset, the best TRA score increases from 0.884 to 0.962, while on U373 the score rises from 0.531 to 0.985. Similar trends are observed across 3T3 and RAW264, demonstrating that higher-resolution features are critical for capturing fine-grained cell dynamics and improving tracking accuracy, especially in challenging datasets with small or densely packed cells. Con-

l_{win}	U373	GOWT1	Huh7	l_{win}	HeLa	PC-3	3T3	RAW264
90	0.957 / 2.9	0.946 / 7.7	0.967 / 3.7	40	0.921 / 1.9	0.938 / 1.6	0.836 / 3.5	0.866 / 3.1
100	0.985 / 2.6	0.960 / 7.1	0.966 / 3.2	50	0.926 / 1.3	0.952 / 1.1	0.854 / 2.2	0.864 / 2.8
110	0.983 / 2.5	0.958 / 7.2	0.966 / 3.1	60	0.923 / 1.5	0.950 / 1.4	0.856 / 2.2	0.859 / 3.0

Table 8: **Ablation on Window Size in RAW.** The results indicate TRA values with inference times (TRA/IT). Setting $l_{win} = 50$ for the DeepCell database and $l_{win} = 100$ for the ISBI CTC datasets during RAW inference.

sequently, we adopt $s = 4$ as the default stride setting in the CAP framework.

Ablation on Window Size. Table 8 reports an ablation of the RAW rolling window size l_{win} , showing TRA together with the inference time (IT). Across the ISBI CTC datasets, $l_{win} = 100$ provides the best overall trade-off: it yields the highest TRA on U373 and GOWT1 while also achieving the fastest or near-fastest inference (2.6s and 7.1s, respectively), and it remains essentially tied on Huh7 at $l_{win} = 90$. On the DeepCell database, $l_{win} = 50$ similarly offers the most favorable balance, delivering the top TRA on HeLa and PC-3 with the lowest inference times, while matching or nearly matching the best results on 3T3 with comparable IT and RAW264. Based on these observations, we set $l_{win} = 100$ for ISBI CTC and $l_{win} = 50$ for DeepCell in all experiments.

5. Conclusion and Future Work

In this paper, we introduce the one-stage CAP (Cell as Point) framework, which eliminates the need for a separate segmentation or detection stage, a significant innovation in cell tracking. This approach substantially improves efficiency via reducing inference time by a factor of 8 to 32 and minimizing the need for high-quality data annotation. CAP effectively captures complex cell states and tracks cells in one stage while maintaining high efficiency. Additionally, it addresses common challenges in cell sequence data, such as cell event imbalance and long sequence inference. In conclusion, CAP not only offers considerable practical value in real-world applications due to its efficiency but also pioneers a novel approach in the field of cell tracking.

Limitations and Future Work. The limitation of our study arises from the nature of the datasets used for evaluation. These datasets were annotated by different experts, and variations in annotation practices may lead to differences in the location of the cell points that CAP generated. Such inconsistencies can introduce discrepancies in performance across datasets, as the model could be sensitive to variations in label quality and annotation style. In addition, differences in imaging devices and cell densities across datasets may further amplify these effects, contributing to dataset-specific performance variation. Consequently, while CAP demonstrates strong trade-offs between accuracy and efficiency, improving generalization remains important for practical deployment. In future work, we will target generalization across: (i) cross-lab/cross-microscope domain shifts, including different sensors, optics, staining/fluorescence conditions, and illumination changes; (ii) cell populations with substantially different morphology and motion statistics, such as highly deformable cells or fast-moving immune cells; (iii) challenging crowding regimes, including very high cell density, frequent occlusions, and long-term interactions where identity switches are common; and (iv) varying acquisition protocols, e.g., different frame rates, missing frames, and longer sequences.

References

- [1] N. Emami, Z. Sedaei, R. Ferdousi, Computerized cell tracking: Current methods, tools and challenges, *Visual Informatics* (2021) 1–13.
- [2] K. Li, E. D. Miller, M. Chen, T. Kanade, L. E. Weiss, P. G. Campbell, Cell population tracking and lineage construction with spatiotemporal context, *Medical Image Analysis* (2008) 546–566.
- [3] M. Rodriguez-Porcel, M. W. Kronenberg, T. D. Henry, J. H. Traverse, C. J. Pepine, S. G. Ellis, J. T. Willerson, L. A. Moyé, R. D. Simari, Cell tracking and the development of cell-based therapies: a view from the cardiovascular cell therapy research network, *JACC: Cardiovascular Imaging* (2012) 559–565.
- [4] S. A. Eccles, C. Box, W. Court, Cell migration/invasion assays and their application in cancer drug discovery, *Biotechnology Annual Review* (2005) 391–421.

- [5] E. Gómez-de Mariscal, H. Grobe, J. W. Pylvänäinen, L. Xénard, R. Henriques, J.-Y. Tinevez, G. Jacquemet, Celltrackscolab is a platform that enables compilation, analysis, and exploration of cell tracking data, *Plos Biology* (2024) e3002740.
- [6] B. Gallusser, M. Weigert, Trackastra: Transformer-based cell tracking for live-cell microscopy, in: *European Conference on Computer Vision*, Springer, 2024, pp. 467–484.
- [7] E. Moen, E. Borba, G. Miller, M. Schwartz, D. Bannon, N. Koe, I. Camplisson, D. Kyme, C. Pavelchek, T. Price, et al., Accurate cell tracking and lineage construction in live-cell imaging experiments with deep learning, *Biorxiv* (2019) 803205.
- [8] K. Löffler, R. Mikut, Embedtrack—simultaneous cell segmentation and tracking through learning offsets and clustering bandwidths, *IEEE Access* (2022) 77147–77157.
- [9] F. A. G. Pena, P. D. M. Fernandez, P. T. Tarr, T. I. Ren, E. M. Meyerowitz, A. Cunha, J regularization improves imbalanced multiclass segmentation, in: *2020 IEEE 17th International Symposium on Biomedical Imaging*, IEEE, 2020, pp. 1–5.
- [10] M. Maška, V. Ulman, P. Delgado-Rodriguez, E. Gómez-de Mariscal, T. Nečasová, F. A. Guerrero Peña, T. I. Ren, E. M. Meyerowitz, T. Scherr, K. Löffler, et al., The cell tracking challenge: 10 years of objective benchmarking, *Nature Methods* (2023) 1010–1020.
- [11] J. Hayashida, K. Nishimura, R. Bise, Mpm: Joint representation of motion and position map for cell tracking, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 3823–3832.
- [12] J. Hayashida, K. Nishimura, R. Bise, Consistent cell tracking in multi-frames with spatio-temporal context by object-level warping loss, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 1727–1736.

- [13] Z. Li, C.-Z. Lu, J. Qin, C.-L. Guo, M.-M. Cheng, Towards an end-to-end framework for flow-guided video inpainting, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 17562–17571.
- [14] S. Wolf, M. Lalit, K. McDole, J. Funke, Unsupervised learning of object-centric embeddings for cell instance segmentation in microscopy images, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 21263–21272.
- [15] M. Chen, Chapter 5 - cell tracking in time-lapse microscopy image sequences, in: M. Chen (Ed.), Computer Vision for Microscopy Image Analysis, Computer Vision and Pattern Recognition, Academic Press, 2021, pp. 101–129.
- [16] J. Stegmaier, R. Mikut, Fuzzy-based propagation of prior knowledge to improve large-scale image analysis pipelines, Plos One (2017) e0187535.
- [17] R. Hoseinnezhad, B.-N. Vo, B.-T. Vo, D. Suter, Visual tracking of numerous targets via multi-bernoulli filtering of image data, Pattern Recognition (2012) 3625–3635.
- [18] O. Hirose, S. Kawaguchi, T. Tokunaga, Y. Toyoshima, T. Teramoto, S. Kuge, T. Ishihara, Y. Iino, R. Yoshida, Spf-celltracker: Tracking multiple cells with strongly-correlated moves using a spatial particle filter, IEEE/ACM Transactions on Computational Biology and Bioinformatics (2017) 1822–1831.
- [19] K. E. G. Magnusson, J. Jaldén, P. M. Gilbert, H. M. Blau, Global linking of cell tracks using the viterbi algorithm, IEEE Transactions on Medical Imaging (2015) 911–929.
- [20] K. E. Magnusson, J. Jaldén, Tracking of non-brownian particles using the viterbi algorithm, in: 2015 IEEE 12th International Symposium on Biomedical Imaging, 2015, pp. 380–384.
- [21] A. Arbelle, J. Reyes, J.-Y. Chen, G. Lahav, T. R. Raviv, A probabilistic approach to joint cell tracking and segmentation in high-throughput microscopy videos, Medical Image Analysis (2018) 140–152.

- [22] E. Türetken, X. Wang, C. J. Becker, C. Haubold, P. Fua, Network flow integer programming to track elliptical cells in time-lapse sequences, *IEEE Transactions on Medical Imaging* (2016) 942–951.
- [23] K. Löffler, T. Scherr, R. Mikut, A graph-based cell tracking algorithm with few manually tunable parameters and automated segmentation error correction, *Plos One* (2021) e0249257.
- [24] Z. Zhou, F. Wang, W. Xi, H. Chen, P. Gao, C. He, Joint multi-frame detection and segmentation for multi-cell tracking, in: *International Conference on Image and Graphics*, Springer, 2019, pp. 435–446.
- [25] J. Hayashida, R. Bise, Cell tracking with deep learning for cell detection and motion estimation in low-frame-rate, in: *Medical Image Computing and Computer-Assisted Intervention*, Springer, 2019, pp. 397–405.
- [26] H.-W. Jung, S.-H. Lee, M. Donnelley, D. Parsons, V. Stamatescu, I. Lee, Multiple particle tracking in time-lapse synchrotron x-ray images using discriminative appearance and neighbouring topology learning, *Pattern Recognition* (2019) 485–497.
- [27] K. Nishimura, J. Hayashida, C. Wang, D. F. E. Ker, R. Bise, Weakly-supervised cell tracking via backward-and-forward propagation, in: *Proceedings of the European Conference on Computer Vision*, Springer, 2020, pp. 104–121.
- [28] L. N. Kirsten, C. R. Jung, Cell tracking-by-detection using elliptical bounding boxes, *Journal of Visual Communication and Image Representation* (2025) 104425.
- [29] R. Bensch, O. Ronneberger, Cell segmentation and tracking in phase contrast images using graph cut with asymmetric boundary costs, in: *2015 IEEE 12th International Symposium on Biomedical Imaging*, 2015, pp. 1220–1223.
- [30] S. U. Akram, J. Kannala, L. Eklund, J. Heikkilä, Joint cell segmentation and tracking using cell proposals, in: *2016 IEEE 13th International Symposium on Biomedical Imaging*, 2016, pp. 920–924.

- [31] R. Bao, N. M. Al-Shakarji, F. Bunyak, K. Palaniappan, Dmnet: Dual-stream marker guided deep network for dense cell segmentation and lineage tracking, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 3361–3370.
- [32] T. Ben-Haim, T. Riklin-Raviv, Graph neural network for cell tracking in microscopy videos, in: *Proceedings of the European Conference on Computer Vision*, Springer, 2022, pp. 610–626.
- [33] T. Scherr, K. Löffler, M. Böhlend, R. Mikut, Cell segmentation and tracking using cnn-based distance predictions and a graph-based matching strategy, *Plos One* (2020) e0243219.
- [34] O. M. O’Connor, M. J. Dunlop, Cell-tractr: A transformer-based model for end-to-end segmentation and tracking of cells, *PLOS Computational Biology* (2025) e1013071.
- [35] M. Rempfler, S. Kumar, V. Stierle, P. Paulitschke, B. Andres, B. H. Menze, Cell lineage tracing in lens-free microscopy videos, in: *Medical Image Computing and Computer-Assisted Intervention*, Springer, 2017, pp. 3–11.
- [36] C. Payer, D. Štern, T. Neff, H. Bischof, M. Urschler, Instance segmentation and tracking with cosine embeddings and recurrent hourglass networks, in: *International Conference on Medical Image Computing and Computer-Assisted Intervention*, Springer, 2018, pp. 3–11.
- [37] H. Yang, Z. Wang, X. Liu, C. Li, J. Xin, Z. Wang, Deep learning in medical image super resolution: a review, *Applied Intelligence* (2023) 20891–20916.
- [38] C. Doersch, A. Gupta, L. Markeeva, A. Recasens, L. Smaira, Y. Aytar, J. Carreira, A. Zisserman, Y. Yang, TAP-vid: A benchmark for tracking any point in a video, *Advances in Neural Information Processing Systems* (2022) 13610–13626.
- [39] A. W. Harley, Z. Fang, K. Fragkiadaki, Particle video revisited: Tracking through occlusions using point trajectories, in: *Proceedings of the European Conference on Computer Vision*, Springer, 2022, pp. 59–75.

- [40] Y. Zheng, A. W. Harley, B. Shen, G. Wetzstein, L. J. Guibas, Pointodyssey: A large-scale synthetic dataset for long-term point tracking, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 19855–19865.
- [41] Q. Wang, Y.-Y. Chang, R. Cai, Z. Li, B. Hariharan, A. Holynski, N. Snavely, Tracking everything everywhere all at once, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 19795–19806.
- [42] N. Karaev, I. Rocco, B. Graham, N. Neverova, A. Vedaldi, C. Rupprecht, Co-Tracker: It is better to track together, in: Proceedings of the European Conference on Computer Vision, 2024, pp. 18–35.
- [43] Z. Teed, J. Deng, Raft: Recurrent all-pairs field transforms for optical flow, in: Proceedings of the European Conference on Computer Vision, Springer, 2020, pp. 402–419.
- [44] N. Karaev, Y. Makarov, J. Wang, N. Neverova, A. Vedaldi, C. Rupprecht, Co-tracker3: Simpler and better point tracking by pseudo-labelling real videos, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2025, pp. 6013–6022.
- [45] V. Ulman, M. Maška, K. E. Magnusson, O. Ronneberger, C. Haubold, N. Harder, P. Matula, P. Matula, D. Svoboda, M. Radojevic, et al., An objective comparison of cell-tracking algorithms, *Nature Methods* (2017) 1141–1152.
- [46] D. F. E. Ker, S. Eom, S. Sanami, R. Bise, C. Pascale, Z. Yin, S.-i. Huh, E. Osuna-Highley, S. N. Junkers, C. J. Helfrich, et al., Phase contrast time-lapse microscopy datasets with automated and manual cell tracking annotations, *Scientific Data* 5 (1) (2018) 1–12.
- [47] S. Anjum, Ctmc: Cell tracking with mitosis detection dataset challenge, in: CVPR Workshops, 2020, pp. 982–983.
- [48] B. Krawczyk, Learning from imbalanced data: open challenges and future directions, *Progress in Artificial Intelligence* (2016) 221–232.

- [49] S. Inoué, Cell division and the mitotic spindle, *The Journal of Cell Biology* (1981) 131s–147s.
- [50] M. Babaeizadeh, M. T. Saffar, S. Nair, S. Levine, C. Finn, D. Erhan, Fitvid: Overfitting in pixel-level video prediction, *arXiv preprint arXiv:2106.13195* (2020).
- [51] N. Yudistira, M. Kavitha, T. Itabashi, A. H. Iwane, T. Kurita, Prediction of sequential organelles localization under imbalance using a balanced deep u-net, *Scientific Reports* (2020) 2626.
- [52] J. Liu, M. Lu, K. Chen, X. Li, S. Wang, Z. Wang, E. Wu, Y. Chen, C. Zhang, M. Wu, Overfitting the data: Compact neural video delivery via content-aware feature modulation, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 4631–4640.
- [53] Y. Liu, S. Wang, Y. Q. Chen, Automatic 3d tracking system for large swarm of moving objects, *Pattern Recognition* (2016) 384–396.
- [54] P. Matula, M. Maška, D. V. Sorokin, P. Matula, C. Ortiz-de Solórzano, M. Kozubek, Cell tracking accuracy measurement based on comparison of acyclic oriented graphs, *Plos One* (2015) e0144959.
- [55] A. Kan, C. Leckie, J. Bailey, J. Markham, R. Chakravorty, Measures for ranking cell trackers without manual validation, *Pattern Recognition* (2013) 2849–2859.